영지식증명 톺아보기 4편: Zcash Sprout - 최초의 프라이버시 암호화폐 (2016)

Author: 김대익
Reviewed By: —

Disclaimer: 본 글은 Zcash의 첫 번째 버전인 Sprout(2016)을 다룹니다. Trusted Setup의 의미와 한계, 보안 취약점, 그리고 이후 발전 방향을 분석합니다.

---

1. 들어가며: ZKP가 실제로 작동하다

2016년 10월 28일, **Zcash(ZEC)**가 출시되었습니다[^1].

이것은 단순한 또 하나의 암호화폐가 아니었습니다. **영지식증명이 실제 프로덕션에서 작동하는 최초의 사례**였습니다.

Zcash Sprout이 증명한 것:

- zk-SNARK가 이론이 아닌 **현실에서 작동**한다
- 프라이버시가 **기술적으로 보장**될 수 있다
- Trusted Setup이 **현실적으로 가능**하다

하지만 Sprout은 완벽하지 않았습니다:

- 증명 생성에 **40초, 3GB 메모리** 필요
- **6명의 참여자**에 의존하는 Trusted Setup
- 나중에 발견된 **심각한 보안 취약점** (CVE-2019-7167)

이번 편에서는 Zcash Sprout의 기술적 구조, Trusted Setup의 의미, 그리고 그 한계와 교훈을 다룹니다.

[^1]: Zcash Blog. (2016). *Zcash Launch and Roadmap*. https://github.com/zcash/zcash-blog/blob/master/_posts/2016-08-17-zcash-launch-and-roadmap.md

---

2. Zcash의 탄생 배경

2.1 비트코인의 프라이버시 문제

비트코인은 **가명(pseudonymous)**이지, **익명(anonymous)**이 아닙니다.

모든 거래가 블록체인에 영구적으로 기록됩니다:

- 보내는 주소
- 받는 주소
- 금액
- 시간

주소와 실제 신원이 한 번이라도 연결되면 (거래소 KYC 등), 모든 거래 이력이 추적 가능합니다.

2.2 Zerocoin에서 Zcash로

Zcash의 원형은 **Zerocoin** (2013)입니다. Johns Hopkins 대학의 매튜 그린(Matthew Green) 교수와 그의 학생들이 제안한 비트코인 확장 프로토콜이었습니다.

하지만 Zerocoin은 한계가 있었습니다:

- 고정된 금액만 숨길 수 있음
- 송신자만 숨김 (수신자, 금액은 공개)
- 비트코인 통합의 어려움

이를 해결하기 위해 **Zerocash** 논문이 발표되었고 (2014), 이것이 Zcash의 기반이 되었습니다.

2.3 핵심 혁신: zk-SNARK 적용

Zcash는 zk-SNARK를 사용하여:

- **송신자** 숨김
- **수신자** 숨김
- **금액** 숨김
- 그러면서도 **이중 지불 방지** (zero-knowledge로 증명)

이것을 가능하게 한 것이 **BCTV14** 증명 시스템입니다.

---

3. BCTV14: Sprout의 증명 시스템

3.1 BCTV14란?

BCTV14는 Ben-Sasson, Chiesa, Tromer, Virza가 2014년에 발표한 zk-SNARK 구성입니다[^2]:

> "Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture"
> USENIX Security 2014

이 논문은 Pinocchio (PGHR13)를 확장하여 TinyRAM이라는 가상 머신 위에서 임의의 프로그램을 증명할 수 있게 했습니다.

Zcash Sprout은 BCTV14의 R1CS 기반 zkSNARK를 사용했으며, **libsnark** 라이브러리로 구현되었습니다[^3].

[^2]: Ben-Sasson, E., et al. (2014). *Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture*. USENIX Security 2014. https://eprint.iacr.org/2013/879 [^3]: GitHub. *zcash/libsnark*. https://github.com/zcash/libsnark

3.2 Sprout의 구성 요소

타원곡선: BN128 (BN254)

- 약 254비트 필드
- 페어링 친화적 커브
- **약 100비트 보안** (128비트 미만, 후에 문제로 지적됨)

증명 시스템: BCTV14 기반 ppzkSNARK

- Preprocessing zkSNARK (사전에 파라미터 생성 필요)
- R1CS → QAP 변환
- Pinocchio 프로토콜 기반

구현: libsnark (C++)

- SCIPR Lab에서 개발
- gadgetlib1/gadgetlib2로 회로 구성

3.3 JoinSplit: 핵심 트랜잭션 구조

Sprout의 프라이버시 트랜잭션은 **JoinSplit**이라는 구조를 사용합니다:

```
입력 (최대 2개 shielded notes) → [ZK Proof] → 출력 (최대 2개 shielded notes)
```

JoinSplit이 증명하는 것:

1. 입력 노트들이 실제로 존재한다 (Merkle 트리 멤버십)
2. 입력 노트들의 소유권이 있다 (spending key 알고 있음)
3. 입력 노트들이 이미 사용되지 않았다 (nullifier가 새것)
4. 입력 합계 = 출력 합계 (잔액 보존)

이 모든 것을 **입력 노트, 출력 노트, 금액을 공개하지 않고** 증명합니다.

3.4 성능 한계

Sprout의 가장 큰 문제는 **성능**이었습니다[^4]:

지표	Sprout (2016)
증명 생성 시간	~40초
메모리 요구량	3GB+
증명 크기	~300 bytes
검증 시간	~10ms

40초와 3GB의 메모리는:

- **모바일 기기에서 사용 불가**
- 일반 사용자에게 너무 느림
- 결과적으로 대부분의 트랜잭션이 **투명(transparent)** 주소 사용

[^4]: Electric Coin Company. (2018). *What's New in Sapling*. https://electriccoin.co/blog/whats-new-in-sapling/

---

4. Trusted Setup: "The Ceremony"

4.1 왜 Trusted Setup이 필요한가

zk-SNARK는 **공통 참조 문자열(Common Reference String, CRS)**을 필요로 합니다.

CRS 생성 과정에서 비밀 값 τ (tau)가 사용됩니다:

```
[G, τG, τ²G, τ³G, ..., τⁿG]
```

이 τ를 알면 **가짜 증명을 만들 수 있습니다**. Zcash에서는 이것이 **화폐 위조**를 의미합니다.

τ를 "**Toxic Waste (독성 폐기물)**"라고 부르는 이유입니다[^5].

[^5]: Electric Coin Company. (2016). *The Design of the Ceremony*. https://electriccoin.co/blog/the-design-of-the-ceremony/

4.2 Multi-Party Computation (MPC)

해결책: **다자간 연산(Multi-Party Computation)**

여러 참여자가 각각 τ의 조각을 생성하고, 조각들을 결합합니다:

```
τ = τ₁ × τ₂ × τ₃ × ... × τ₆
```

핵심 보안 가정:

> "참여자 중 **단 한 명이라도** 자신의 조각을 파괴하면, 전체 τ는 영원히 복구 불가능하다."

이것이 "**1-of-n 신뢰 가정**"입니다.

4.3 Sprout 세레모니 (2016년 10월 22-23일)

Sprout의 Trusted Setup은 **6명의 참여자**로 진행되었습니다[^6]:

1. **Zooko Wilcox** - Zcash CEO
2. **Andrew Miller** - 암호학 연구자
3. **Peter Van Valkenburgh** - Coin Center
4. **Peter Todd** - 비트코인 개발자
5. **Derek Hinch** - 익명 참여자
6. **Edward Snowden** (당시 익명, 나중에 공개) - NSA 내부고발자

[^6]: GitHub. *zcash/mpc*. https://github.com/zcash/mpc

4.4 세레모니의 보안 조치

Peter Todd의 상세한 기록에 따르면[^7]:

물리적 보안:

- Air-gapped 컴퓨터 사용 (네트워크 완전 차단)
- 새 컴퓨터, 새 DVD로 부팅
- "Cypherpunk Desert Bus" - 인적이 드문 장소로 이동하며 진행

절차적 보안:

- 각 참여자가 독립적으로 진행
- 비디오 녹화로 증거 보존
- 세레모니 후 컴퓨터 물리적 파괴

한계:

- 6명 모두가 공모하면 위조 가능
- 참여자를 신뢰해야 함
- **증명 불가능**: "정말로 파괴했는가?"를 제3자가 검증할 방법이 없음

[^7]: Peter Todd. (2016). *Cypherpunk Desert Bus: My Role In The 2016 Zcash Trusted Setup Ceremony*. https://petertodd.org/2016/cypherpunk-desert-bus-zcash-trusted-setup-ceremony

4.5 Zooko의 경고

세레모니 직후, Zcash CEO Zooko Wilcox는 솔직하게 말했습니다[^5]:

> "세레모니를 직접 목격하지 않은 사람들은 여전히 의심할 수 있습니다. 우리가 실제로 말한 대로 했다는 것을 증명할 방법이 없습니다. 6명 모두가 공모하여 전체 과정을 연출하고, 비밀리에 τ를 보관했을 수도 있습니다."

이 한계를 인정하면서, 그는 향후 **위조 탐지 기능** 추가를 약속했습니다.

---

5. CVE-2019-7167: 발견된 위조 취약점

5.1 취약점의 발견

2018년 3월 1일, Zcash 암호학자 **Ariel Gabizon**이 BCTV14 논문에서 **미묘한 암호학적 결함**을 발견했습니다[^8].

이 결함은:

- BCTV14 논문의 Trusted Setup 절차에 존재
- **"bypass elements"**라는 추가 요소가 생성됨
- 이 요소를 가진 사람은 **가짜 증명 생성 가능 → 화폐 위조**

[^8]: Electric Coin Company. (2019). *Zcash Counterfeiting Vulnerability Successfully Remediated*. https://electriccoin.co/blog/zcash-counterfeiting-vulnerability-successfully-remediated/

5.2 왜 심각했나

영향 범위:

- BCTV14를 사용하는 모든 시스템
- Sprout MPC 세레모니 트랜스크립트에 bypass elements 포함
- 트랜스크립트에 접근 가능한 누구나 위조 가능

아이러니:

- libsnark 구현에서는 이 요소들이 proving key에서 **제외**됨

- 하지만 MPC 세레모니 트랜스크립트에는 **포함**되어 있었음
- 트랜스크립트는 공개 자료였음

5.3 비밀 수정

Zcash 팀은 **책임감 있는 공개(Responsible Disclosure)** 원칙을 따랐습니다:

비밀리에 진행된 조치:

1. Nathan Wilcox(CTO)가 공개 트랜스크립트를 **삭제** (운영 보안 명목)
2. Sapling 업그레이드에 **비밀리에** 패치 포함
3. 관련 프로젝트(Komodo 등)에 **비공개 통보**

공개 타임라인:

- 2018년 3월: 취약점 발견
- 2018년 10월: Sapling 활성화 (블록 419,200)
- 2019년 2월: **11개월 후** 공개 발표

5.4 교훈

기술적 교훈:

- 논문의 이론적 정확성 ≠ 구현의 보안
- MPC 프로토콜과 증명 시스템의 미묘한 상호작용
- 코드 리뷰만으로는 부족 (수학적 분석 필요)

운영적 교훈:

- 최소 참여자 수의 한계 (6명은 적음)
- 트랜스크립트 공개의 위험성
- 빠른 패치 배포 체계 필요

좋은 소식:

- **위조가 실제로 발생했다는 증거 없음**
- Sapling 이후 Zcash는 이 취약점으로부터 **안전**

---

6. Sprout vs Sapling: 2년의 발전

6.1 성능 비교

Sapling (2018년 10월)은 극적인 개선을 이뤘습니다[^9]:

지표	Sprout (2016)	Sapling (2018)	개선
증명 시간	~40초	~7초	**81% 감소**
메모리	3GB+	~40MB	**97% 감소**
타원곡선	BN128 (~100비트)	BLS12-381 (128비트)	보안 강화

지표	Sprout (2016)	Sapling (2018)	개선
증명 시스템	BCTV14	Groth16	더 효율적
구현	libsnark (C++)	Bellman (Rust)	메모리 안전

[^9]: Messari. (2025). *Understanding Zcash: A Comprehensive Overview*. https://messari.io/report/understanding-zcash-a-comprehensive-overview

6.2 기술적 변화

Groth16 적용:

- 더 짧은 증명 (~192 bytes vs ~300 bytes)
- 더 빠른 검증
- 더 효율적인 회로

새로운 해시 함수:

- SHA-256 → Pedersen Hash
- 타원곡선 위에서 효율적으로 계산
- 회로 크기 대폭 감소

Jubjub 커브:

- BLS12-381 위에 정의된 "embedded curve"
- 서명과 커밋먼트에 최적화

6.3 Powers of Tau

Sapling의 Trusted Setup은 **Powers of Tau** 세레모니로 진행되었습니다[^10]:

- **87명** 참여 (vs Sprout의 6명)
- **누구나 참여 가능** (사전 식별 불필요)
- **순차적 참여** (한 명이 끝나면 다음 참여자)
- 더 길어진 기간 (몇 달에 걸쳐 진행)

유명한 참여 사례:

- Andrew Miller: **비행기 위에서** 참여
- 무작위 소스: **체르노빌 방사성 물질**의 붕괴 측정값을 엔트로피로 사용

[^10]: Messari. (2025). *Understanding Zcash: A Comprehensive Overview*.

---

7. 현재 상태: Sprout의 일몰

7.1 Sprout 풀의 현황

2025년 기준:

- **Sprout**: ~25,600 ZEC (전체 유통량의 0.2%)
- **Sapling**: ~636,000 ZEC (3.9%)

- **Orchard**: ~4,200,000 ZEC (25.4%)

Sprout 주소 ("zc"로 시작)는:

- 더 이상 권장되지 않음
- zcashd에서만 지원 (deprecated 예정)
- Sapling으로 이전 권고

7.2 Turnstile 메커니즘

Sprout 취약점 패치 후, **Turnstile(회전문)** 메커니즘이 도입되었습니다:

- Sprout ↔ Sapling 직접 전환 불가
- 반드시 **투명 주소를 경유**해야 함
- 위조된 ZEC가 있어도 Sprout 풀에 **격리**됨

이것은 "손상 제어(Damage Control)" 전략입니다.

7.3 Halo2와 Trusted Setup의 종말

2022년, Zcash는 **NU5 (Network Upgrade 5)**와 함께 **Orchard 풀**을 도입했습니다.

Orchard는 **Halo2** 증명 시스템을 사용합니다:

- **Trusted Setup 불필요** (trustless!)
- 재귀적 증명 가능
- PLONK 기반 arithmetization

이로써 Zcash는 더 이상 Trusted Setup에 의존하지 않게 되었습니다.

---

8. 교훈: Sprout이 남긴 것

8.1 증명된 것

ZKP의 실용성:

- 이론이 현실에서 작동
- 대규모 사용자 기반에서 검증됨
- 암호학적 프라이버시가 가능함

Trusted Setup의 가능성:

- MPC로 신뢰 분산 가능
- 1-of-n 보안 모델 실증
- (하지만 한계도 명확)

8.2 드러난 한계

성능:

- 3GB 메모리, 40초 증명 → 대중 채택 불가
- 모바일/IoT 불가능

신뢰:

- 6명 참여자는 너무 적음
- 트랜스크립트 관리 실수 (취약점 노출)
- "증명 불가능" 문제 해결 안 됨

복잡성:

- 회로 설계의 어려움
- 암호학적 버그 발견의 어려움
- 긴 개발 주기

8.3 후속 발전을 이끈 것

Sprout의 한계가 다음을 촉발:

Sprout의 문제	해결책
느린 증명	Groth16, PLONK
큰 메모리	최적화된 회로, Pedersen Hash
Trusted Setup	Halo, 재귀적 증명
적은 참여자	Powers of Tau (수백 명)
BN128 보안	BLS12-381

---

9. 마치며: 첫 발자국의 의미

Zcash Sprout은 **최초**였기 때문에 중요합니다.

- 최초의 zk-SNARK 프로덕션 배포
- 최초의 대규모 Trusted Setup 세레모니
- 최초의 완전한 거래 프라이버시 (송신자, 수신자, 금액)

그리고 그 한계가 다음 세대의 발전을 이끌었습니다:

- Sapling의 Groth16
- Orchard의 Halo2
- 그리고 더 넓은 ZKP 생태계 (zkSync, Scroll, Polygon zkEVM, ...)

다음 편에서는 **Groth16**을 심층적으로 다룹니다:

- Trusted Setup의 수학적 구조
- 페어링 기반 검증
- 왜 증명이 3개의 점으로 충분한가
- KZG 커밋먼트의 원리

시리즈 로드맵

1편: 비밀을 증명한다는 것 ✓
2편: 컴퓨터가 이해하는 수학 ✓
3편: 프로그램을 다항식으로 - 산술화 ✓
4편: Zcash Sprout - 최초의 프라이버시 암호화폐 (현재) ✓
5편: Groth16의 마법
6편: Filecoin의 PoRep - 저장 증명
...

References

1. GitHub. (2016). *Zcash Launch and Roadmap*. https://github.com/zcash/zcash-blog/blob/master/_posts/2016-08-17-zcash-launch-and-roadmap.md
2. Ben-Sasson, E., Chiesa, A., Tromer, E., & Virza, M. (2014). *Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture*. USENIX Security 2014. https://eprint.iacr.org/2013/879
3. GitHub. *zcash/libsnark*. https://github.com/zcash/libsnark
4. Electric Coin Company. (2018). *What's New in Sapling*. https://electriccoin.co/blog/whats-new-in-sapling/
5. Electric Coin Company. (2016). *The Design of the Ceremony*. https://electriccoin.co/blog/the-design-of-the-ceremony/
6. GitHub. *zcash/mpc*. https://github.com/zcash/mpc
7. Peter Todd. (2016). *Cypherpunk Desert Bus: My Role In The 2016 Zcash Trusted Setup Ceremony*. https://petertodd.org/2016/cypherpunk-desert-bus-zcash-trusted-setup-ceremony
8. Electric Coin Company. (2019). *Zcash Counterfeiting Vulnerability Successfully Remediated*. https://electriccoin.co/blog/zcash-counterfeiting-vulnerability-successfully-remediated/
9. Messari. (2025). *Understanding Zcash: A Comprehensive Overview*. https://messari.io/report/understanding-zcash-a-comprehensive-overview
10. ZKProof Standards. (2021). *Setup Ceremonies*. https://zkproof.org/2021/06/30/setup-ceremonies/
11. CoinBureau. (2024). *What is the ZCash Ceremony?*. https://coinbureau.com/education/zcash-ceremony/
12. ZKProof Standards. (2021). *Zebra: Zcash Zero-Knowledge Proofs at Scale*. https://zkproof.org/2021/06/03/zebra-zcash-zero-knowledge-proofs-at-scale/